

# چرخه جدید تولید نرم افزار با کدنویسی شهودی

از پرامپت‌های (prompt) نامنظم تا مهندسی عامل محور (Agentic engineering)

بر پایه مقاله آدی عثمانی، شوبهام سابو و سوکراتیس کارتاکیس

## پنج جلسه، یک مسیر

### ۱ – چرخه جدید تولید نرم افزار

چارچوب ذهنی: طیف، زمینه، مهار، نقش تازه توسعه دهنده.

### ۲ – ابزارهای عامل و تعامل پذیری

عامل (agent) چگونه با جهان بیرون حرف می زند: ابزار، MCP، A2A.

### ۳ – مهارت های عامل

نشست، مهارت (skill) و حافظه؛ اقتصاد توکن (token) در مقیاس تولید.

### ۴ – امنیت و ارزیابی

سطح حمله تازه، گارد ریل ها، و سنجش کیفیت عامل.

### ۵ – توسعه مبتنی بر مشخصات در سطح محصول

از نمونه اولیه تا نرم افزاری که می شود به آن تکیه کرد.

## این دیگر آینده پژوهی نیست

۱.۴٪

از کدهای تازه نوشته شده، تولید هوش مصنوعی است

۱.۵٪

هر روز از آن ها استفاده می کنند

۵.۸٪

از توسعه دهندگان حرفه ای مرتب از عامل های کدنویسی استفاده می کنند

مسیر این تحول: تکمیل خودکار ← پیشنهاد درون خطی ← گفت و گو ← عامل خودگردانی که ریپو (repository) را می خواند، تغییر می دهد، تست (test) می گیرد و پول ریکوئست (pull request) می سازد.

عمیق‌ترین تغییر مهندسی نرم‌افزار، یک زبان یا چارچوب تازه نیست؛  
گذار از نوشتن کد به بیان قصد (intent) است – و اعتماد به  
سامانه‌های هوشمند برای ترجمهٔ آن قصد به نرم‌افزار کارآمد.

تذات اصلی جلسهٔ امروز

# عامل چیست؟

یک مرور سریع، چون هر چیز دیگری در این دوره روی همین تعریف بنا می‌شود.

# حلقه عامل

ربات گفت‌وگو پاسخ می‌دهد و منتظر می‌ماند. عامل حلقه خودش را می‌چرخاند: شما هدف را می‌دهید، او در هر گام تصمیم می‌گیرد قدم بعدی چیست.



**نکته:** این حلقه، قلب تپنده هر عامل است. هر چیزی که در چهار جلسه بعد می‌بینید، صورتی از همین حلقه (loop) است.

# پنج جزء سازنده هر عامل

۱

## مدل

موتور استدلال: زمینه (context) را می‌خواند و تصمیم می‌گیرد گام بعد چه باشد.

۲

## ابزارها

اتصال مدل به جهان: API، اجرای کد، پرس‌وجوی پایگاه داده، واگذاری (handoff) به عامل دیگر.

۳

## حافظه

وضعیت: یادآوری تعامل‌های پیشین و قواعد پروژه، تا هرگز از صفر شروع نکند.

۴

## ارکستراسیون

کدی که حلقه را می‌چرخاند: ساختن زمینه، فرستادن فراخوان‌ها، تصمیم به ادامه یا توقف.

۵

## دیپلوی

آنچه نمونه اولیه را به سرویس تبدیل می‌کند: میزبانی، هویت، رصدپذیری (observability) و زیرساخت تولید.

## چرا این تفکیک مهم است؟

| عامل                               | ربات گفت‌وگو     |                      |
|------------------------------------|------------------|----------------------|
| یک هدف، چندین گام                  | یک پرسش، یک پاسخ | واحد کار             |
| خود سامانه                         | انسان            | تصمیم‌گیرنده گام بعد |
| کنش واقعی از راه ابزار             | فقط متن          | نسبت با جهان         |
| رسیدن به هدف یا برخورد به شرط توقف | پایان پاسخ       | شرط پایان            |
| کنش نادرست روی سامانه واقعی        | پاسخ نادرست      | ریسک                 |

**پیامد:** وقتی سامانه می‌تواند کنش کند، «کیفیت پاسخ» کافی نیست؛ به مهار، گاردریل (guardrail) و ارزیابی (evaluation) نیاز داریم.

# طیف: از کدنویسی شهودی تا مهندسی عامل محور

پرسش درست این نیست که «آیا از هوش مصنوعی استفاده می‌کنید؟» بلکه این است که «چقدر ساختار، راستی‌آزمایی و قضاوت انسانی پیرامون خروجی آن هست؟»

# کدنویسی شهودی چیست؟

در فوریهٔ ۲۰۲۵، آندری کارپشی شیوه‌ای را توصیف کرد که خیلی‌ها بی‌آنکه نامی برایش داشته باشند، به آن عمل می‌کردند:

کاملاً تسلیم حس و حال شو، رشد شتابان توانایی‌های هوش مصنوعی را بپذیر و فراموش کن که اصلاً کدی وجود دارد.

- خواسته را به زبان طبیعی توصیف می‌کنی.
- خروجی مدل را می‌پذیری، بدون آنکه لزوماً بخوانی‌اش.
- وقتی چیزی خراب شد، پیام خطا را کپی می‌کنی و می‌گویی «درستش کن».

# واژه‌ای که از فرط گستردگی، معنایش را باخت

## مسئله

ظرف چند ماه، «کدنویسی شهودی (Vibe coding)» به هر گردش کار (workflow) کمک گرفته از هوش مصنوعی گفته شد. آیا مهندس ارشدی که یک ویژگی دقیقاً مشخص شده را با کمک هوش مصنوعی پیاده می‌کند، «شهودی» کار می‌کند؟

## پاسخ

تا اوایل ۲۰۲۶ خود کارپشی پذیرفت که قاب اولیه تنگ بوده و واژه مهندسی عامل محور را برای سر منضبط این طیف پیشنهاد کرد.

**چرا واژه مهم است:** اگر به مدیر فنی بگویید تیم شما سامانه پرداخت را «شهودی» می‌نویسد، باید زنگ خطر به صدا دربیاید – و درست هم هست. اگر بگویید «مهندسی عامل محور» می‌کنید، گفت‌وگو یکسره چیز دیگری است.

# طیف در شش بُعد

| بُعد         | کدنویسی شهودی                     | کمک گرفتن ساختارمند                        | مهندسی عامل محور                                             |
|--------------|-----------------------------------|--------------------------------------------|--------------------------------------------------------------|
| بیان قصد     | پرامپت‌های گفتاری و سرپرستی       | پرامپت‌های دقیق با مثال و قید (constraint) | مشخصات (spec) رسمی، سند معماری، فایل‌های حافظه (memory)      |
| راستی‌آزمایی | «به نظر کار می‌کند؟»              | تست دستی و واری‌نمونه‌ای                   | مجموعه‌تست خودکار، دروازه‌های CI/CD (gate)، داوران مدل‌زبانی |
| خواندن کد    | ممکن است اصلاً خوانده نشود        | بازبینی گزینشی مسیرهای حساس                | بازبینی کامل معماری؛ جزئیات پیاده‌سازی با عامل               |
| خطا          | پیام خطا را برگردان به مدل        | انسان ریشه را می‌یابد، عامل اصلاح می‌کند   | عامل درون مرزهای تعریف‌شده خودتشخیص می‌دهد                   |
| دامنه مناسب  | نمونه اولیه، اسکریپت، هکاتون      | افزودن ویژگی به کدبیس موجود                | سامانه تولیدی، توسعه در مقیاس تیم                            |
| ریسک         | بالا؛ برای کد یک‌بارمصرف پذیرفتنی | متوسط؛ قضاوت انسانی در ایستگاه‌های کلیدی   | پایین؛ راستی‌آزمایی نظام‌مند در هر مرحله                     |

# تمایزدهنده اصلی: چگونه راستی آزمایی می کنید

## ارزیابی ها

بخش غیرقطعی (**not deterministic**) را می سنجند: آیا عامل مسیر (trajectory) درستی رفت؟ ابزار درست را برگزید؟ پاسخ نهایی به حد کیفی رسید؟ داورشان دادگان برچسب خورده، سنجه ها (rubric) و داوران مدل زبانی اند.

## تست ها

بخش قطعی (**deterministic**) سامانه را می سنجند: این ورودی باید آن خروجی را بدهد. داورشان کد است.

**قاعده سخت:** بدون هر دو، کاری که می کنید کدنویسی شهودی است – هر قدر هم درخواست هایتان پیچیده و حرفه ای باشد.

# مهندسی زمینه: مهارت واقعی

کیفیت کد تولیدشده، کمتر به زیرکی درخواست شما بستگی دارد و بیشتر به کیفیت زمینه‌ای که فراهم کرده‌اید وابسته است.

# شش نوع زمینه

## دستورها (Instructions)

نقش عامل، هدف‌ها و مرزهای عملیاتی‌اش.

## دانش (Knowledge)

سندهای بازیابی‌شده، نمودار معماری، داده‌حوزه.

## حافظه (Memory)

کوتاه‌مدت: چه گذشت. بلندمدت: این پروژه چیست.

## مثال‌ها (Examples)

نمونه‌های رفتاری و الگوهای مرجع درون کدبیس.

## ابزارها (Tools)

تعریف دقیق API‌ها، اسکریپت‌ها و سرویس‌های قابل فراخوانی.

## گاردریل‌ها (Guardrails)

قیدهای سخت، قواعد قالب‌بندی، اعتبارسنجی‌های ایمنی.

**پرسش راهنما:** «یک هم‌تیمی تازه‌وارد چه چیزهایی باید بداند تا مفید باشد؟ و چطور آن دانش را به شکلی بنویسم که عامل هم بتواند استفاده کند؟»

# زمینه ایستا در برابر زمینه پویا

## پویا – هنگام نیاز بارگذاری می‌شود

دستور مهارت‌هایی که با کار جاری هم‌خوان‌اند، نتیجه ابزارها، سندهای بازیابی‌شده، بخش اخیر تاریخچه نشست (session).  
هزینه: فقط وقتی می‌پردازید که واقعاً لازم باشد.

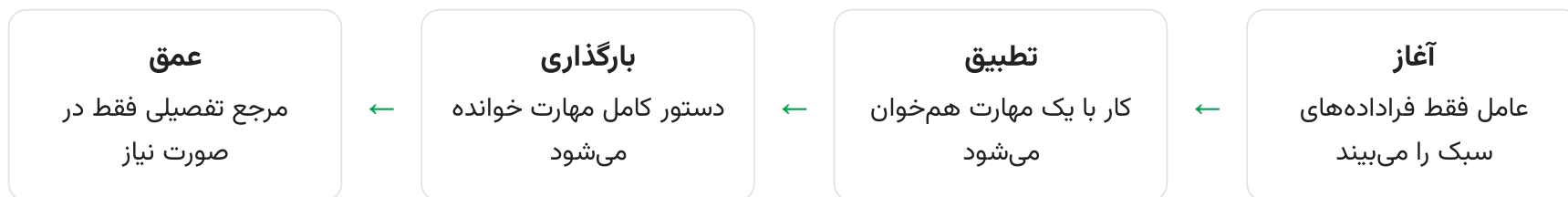
## ایستا – همیشه بارگذاری می‌شود

سیستم پرامپت، فایل‌های قاعده ( AGENTS.md ، CLAUDE.md )، حافظه سراسری، تعریف شخصیت. ( GEMINI.md )  
هزینه: هر توکن آن در هر تعامل حاضر است، چه به کار بیاید چه نه.

**این یک تصمیم معماری است:** زمینه ایستای زیاد، توکن هدر می‌دهد و سیگنال را رقیق می‌کند؛ زمینه ایستای کم، یعنی عامل قواعد حیاتی را فراموش می‌کند. این مرز باید مثل هر پیکربندی دیگری بازبینی و نسخه‌بندی شود.

# مهارت‌های عامل: قوی‌ترین الگوی زمینه پویا

بسته‌های ساختارمند و قابل حمل از دانش رویه‌ای (procedural knowledge)، که عامل فقط وقتی آن‌ها را بارگذاری می‌کند که کار اقتضا کند.



**نتیجه:** عامل می‌تواند ده‌ها توانایی تخصصی حمل کند و فقط برای همانی که همین حالا به کار گرفته، توکن پردازد – یک عامل عمومی سبک که به اقتضا متخصص می‌شود.

# مهارت‌ها چهار درد قدیمی را درمان می‌کنند

## پوسیدگی زمینه

سیستم پرامپت‌هایی که بیش از حد در آن‌ها مطلب گنجانده شده، سیگنال را از بین می‌برند.

## نبود حافظه رویه‌ای

مدل‌های زبانی «چگونه انجامش بدهیم» را به خاطر نمی‌سپارند.

## سربرار معماری چندعاملی

برای هر تخصص، یک عامل جدا ساختن گران و شکننده است.

## نبود قابلیت حمل

دانش تیم نباید به یک ابزار یا فروشنده خاص قفل شود.

**پل به جلسه ۳:** طراحی، نوشتن و ارزیابی مهارت‌ها، مدیریت نشست و حافظه، و اقتصاد توکن را در جلسه سوم به تفصیل می‌بینیم.

# چرخه جدید تولید نرم افزار

وقتی گلوگاه از «دست انسان» به «ذهن انسان» منتقل می شود، خود فرایند باید بازطراحی شود.

# فشرده‌سازی، اما نامتوازن

## آنچه با سرعت انسان می‌ماند

نیازمندی، معماری و راستی‌آزمایی.

## آنچه به شدت کوتاه می‌شود

پیاده‌سازی: کاری که هفته‌ها می‌برد، اکنون ساعت‌ها.

نتیجه، نسخه‌ی سریع‌تر همان چرخه‌ی قدیمی نیست؛ گردش‌کاری متفاوت است که در آن مرز فازها محو می‌شود، تکرارها از هفته به دقیقه می‌رسد، و توسعه‌دهنده از «پیاده‌ساز اصلی» به «طراح سامانه و داور (judge) کیفیت» بدل می‌شود.

# فاز به فاز – نیازمندی و معماری

## نیازمندی و برنامه‌ریزی

- تولید داستان کاربر از خلاصه محصول، کشف حالت‌های مرزی، ساخت اسکیمای API از توضیح زبان طبیعی. حلقه «نیازمندی تا نمونه اولیه» تقریباً به صفر می‌رسد.
- نیازمندی دیگر سندی نیست که بین تیم‌ها دست‌به‌دست شود؛ گفت‌وگویی است که هم‌زمان مشخصات و پیاده‌سازی اولیه تولید می‌کند.

## طراحی و معماری

- سرسخت‌ترین فاز در برابر خودکارسازی؛ چون معماری یعنی موازنه: سازگاری در برابر دسترس‌پذیری، پیچیدگی در برابر انعطاف، ساخت در برابر خرید.
- هوش مصنوعی در اجرای تصمیم معماری عالی است، نه در گرفتن آن.

# فاز به فاز – پیاده‌سازی و تست

## سود بهره‌وری

پیمایش‌های صنعتی ۲۵ تا ۳۹ درصد بهبود گزارش می‌کنند.

## تصویر کامل‌تر

پژوهش METR: توسعه‌دهندگان باتجربه در برخی کارها ۱۹٪ **کندتر** شدند – به‌دلیل وقتِ راستی‌آزمایی و اصلاح خروجی.

هوش مصنوعی کار پیاده‌سازی را حذف نمی‌کند؛ آن را از «نوشتن» به «بازبینی، هدایت و راستی‌آزمایی» تبدیل می‌کند.

**در تست:** هم خروجی را بسنجید (آیا کامپایل می‌شود؟ تست‌ها سبزند؟) هم مسیر را (آیا گام‌های راستی‌آزمایی طی شد؟). خروجی روانی که مسیر را میان‌بر زده، از خروجی با خطای آشکار خطرناک‌تر است.

# فاز به فاز – بازبینی، دیپلوی، نگهداری

- **بازبینی کد:** هوش مصنوعی به عنوان بازبین نوبت اول – یافتن اشکال، نقض سبک، آسیب پذیری و مسائل کارایی پیش از آنکه انسان کد را ببیند. جای انسان را نمی گیرد، بار شناختی او را کم می کند.
- **دیپلوی (deploy):** عامل ها سلامت انتشار را رصد می کنند، انتشار مشکل دار را برمی گردانند و ریسک را بر اساس دامنه تغییر پیش بینی می کنند.
- **نگهداری – کم تخمین زده ترین تحول:** کد بیس هایی که «دست زدن به آن ها پرریسک» بود، حالا قابل فهم و بازسازی اند. مهاجرت چارچوب، به روزرسانی API های منسوخ و نوسازی مجموعه تست ها – کارهایی که پیش تر هرگز انجام نمی شدند.

**جلسه ۵:** وقتی حجم پول ریکوئست ها با خروجی عامل ها مقیاس می گیرد، بازبینی انسانی چطور باید تغییر کند.

# مدل کارخانه

خروجی اصلی توسعه‌دهنده دیگر کد نیست؛ سامانه‌ای است که کد تولید می‌کند.

## مشخصات و زمینه

تعریف آنچه باید ساخته شود.

## عامل‌ها

ترجمه مشخصات به پیاده‌سازی.

## تست‌ها و دروازه‌های کیفیت

راستی‌آزمایی درستی.

## حلقه‌های بازخورد

بازگرداندن شکست‌ها به عامل برای اصلاح.

## گاردریل‌ها

محدود کردن عامل به رفتار ایمن و قابل‌پیش‌بینی.

**قاعده:** مدیر کارخانه هر قطعه را دستی نمی‌سازد؛ خط تولید را طراحی و کنترل کیفیت را تضمین می‌کند. به عامل‌ها «معیار موفقیت» بدهید، نه دستورالعمل گام‌به‌گام.

# مهندسی مهار

اگر توسعه‌دهنده مدیر کارخانه است، مدل فقط موتور روی کف کارگاه است. موتور به‌تنهایی خودرو نمی‌سازد.

## عامل = مدل + مهار

وسوسه‌ای هست که مدل را با سامانه یکی بگیریم: مدل تازه می‌آید، عامل باهوش می‌شود؛ مدل قدیمی، عامل ضعیف. این شهود غلط است و به سرمایه‌گذاری‌های اشتباه می‌انجامد.

مدل خام، عامل نیست. وقتی عامل می‌شود که **مهار (harness)** به آن وضعیت، اجرای ابزار، حلقهٔ بازخورد و قیدهای قابل‌اعمال بدهد.

رفتاری که هنگام کار با Claude Code، Cursor، Codex یا Cline تجربه می‌کنید، بیش از آنکه محصول مدل زیرین باشد، محصول مهار است.

# داخل مهار چیست؟

## دستورها و فایل‌های قاعده

، فایل‌های مهارت، `AGENTS.md`  
درخواست‌های زیرعامل‌ها (subagent):  
عامل کیست و چه چیزی ممنوع است.

## ابزارها

توابع، سرورهای MCP و APIها — به‌علاوه  
نثری که می‌گوید کی و چگونه صدایشان  
بزنند.

## سندباکس و محیط اجرا

کد عامل کجا اجرا می‌شود، به چه چیزی  
دسترسی دارد و به چه چیزی نمی‌رسد.

## منطق ارکستراسیون

زایش زیرعامل، مسیریابی (routing) مدل،  
تحویل کار بین متخصص‌ها و قواعد  
فعال‌شدن هرکدام.

## گاردریل‌ها و قلاب‌ها

کد قطعی که در نقاط مشخص چرخه عمر  
اجرا می‌شود: پیش از فراخوان ابزار، پس از  
ویرایش فایل، پیش از کامیت (commit).

## رصدپذیری

گزارش، ردیابی، ارزیابی، سنجش هزینه و  
تأخیر (latency). بدون آن نمی‌دانیم عامل  
خوب کار می‌کند یا بی‌سروصدا منحرف  
شده.

**توجه:** این سطح گسترده، سطح **تیم شماست**، نه فروشنده مدل.

# مهار در چهار فاز چرخه عمر

| فاز                                       | اجزای فعال مهار              | کاری که انجام می‌شود                                                                                                        |
|-------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| نیازمندی، برنامه، معماری<br>پیکربندی مهار | دستورها و فایل‌های قاعده     | تعریف قیده‌های معماری، مشخص کردن ابزارهای در دسترس، و قواعدی که عامل حق شکستن‌شان را ندارد.                                 |
| پیاده‌سازی<br>اجرای مهار                  | سندباکس، محیط اجرا، ابزارها  | کد درون سندباکس (sandbox) ایزوله اجرا می‌شود؛ خواندن فایل یا جست‌وجو فقط از راه ابزارهای مهار.                              |
| تست و تضمین کیفیت<br>حلقه بازخورد         | منطق ارکستراسیون، گاردریل‌ها | تست در محیط اجرا می‌رود؛ اگر شکست بخورد، ارکستراسیون (orchestration) خطا را می‌گیرد و به مدل برمی‌گرداند: «دوباره تلاش کن». |
| بازبینی، دیپلوی، نگهداری<br>رصد مهار      | قلاب‌ها، رصدپذیری            | قلاب (hook) قطعی جلوی کامیتِ گذرواژه سخت‌کدشده را می‌گیرد؛ لایه رصد هزینه، تأخیر و انحراف (drift) را دنبال می‌کند.          |

## اثر مهار، عددی است

# +13.7

پژوهشی در LangChain با دستکاری فقط سیستم پرامپت،  
ابزارها و میان افزار – روی مدلی ثابت – امتیاز را ۱۳/۷ واحد بالا  
برد.

# Top 30 → Top 5

در بنچمارک Terminal Bench 2.0 (benchmark)، یک تیم  
فقط با تغییر مهار – بدون هیچ تغییری در مدل – از بیرون  
سی‌تای برتر به پنج‌تای برتر رسید.

تفاوت «کدنویسی شهودی» و «مهندسی عامل‌محور» در ابزار نیست؛ یک توسعه‌دهنده می‌تواند با عامل یکسان،  
هر دو کار را بکند. تفاوت در این است که مهار را چقدر آگاهانه پیکربندی کرده‌اید.

وقتی عامل کار اشتباهی می‌کند، اولین گزینه سرزنش مدل است. اما اغلب، رد خطا به یک ابزار غایب، یک قاعدهٔ مبهم، یک گاردریل نبوده یا کانتکست ویندوای پر از نویز می‌رسد.

بیشتر شکست‌های عامل، اگر صادقانه بررسی شوند، شکست پیکربندی‌اند.

# نقش در حال تحول توسعه دهنده

دو حالت کاری که توسعه دهندگان روان بینشان جابه جا می شوند.

# رهبر ارکستر: هدایت همزمان و دستی

## چه شکلی است

در ویرایشگر، همزمان با ظاهرشدن کد، با درخواست و اصلاح هدایتش می‌کنی. کنترل ریزدانه روی هر چیزی که نوشته می‌شود.

## کی مناسب است

منطق پیچیده، دیباگ (debug) دشوار، یا کدبیزی ناآشنا که باید هر تغییر را همان لحظه بفهمی.

## ابزارهای نمونه

Cursor، Gemini Code Assist، GitHub Copilot  
Windsurf.

## ریسک

اگر همه فشارِ کلید را شخصاً هدایت کنی، خودت گلوگاه می‌شوی و بهره سرعت محدود می‌ماند.

# هماهنگ‌کننده: واگذاری ناهم‌زمان به چند عامل

## چه شکلی است

هدف را تعریف می‌کنی، به عامل‌ها می‌سپاری و نتیجه را بازبینی می‌کنی. عامل‌ها در پس‌زمینه، موازی، روی بخش‌های مختلف کدبیس کار می‌کنند.

## کی مناسب است

کارهای خوش‌تعریف: رفع اشکال شناخته‌شده، پیاده‌سازی ویژگی روی الگوی موجود، مهاجرت کدبیس، تولید مجموعه‌تست.

مهارت لازم عوض می‌شود: به‌جای تسلط عمیق بر نحو و اصطلاحات زبان، چهار توانایی دیگر تعیین‌کننده می‌شوند.

## مشخصه‌نویسی

تعریف کار آن‌قدر دقیق که ابهامی نماند.

## تجزیه

شکستن کار بزرگ به واحدهای هم‌اندازهٔ عامل.

## ارزیابی

سنجش سریع اینکه خروجی به حد کیفی رسیده یا نه.

## طراحی سامانه

ساختن قیدها، تست‌ها و حلقه‌های بازخوردی که عامل را بارور نگه می‌دارد.

# عوامل‌های کدنویسی در روزِ کاری، سه جا ظاهر می‌شوند

## در پس‌زمینه

کار را می‌گیرد، ساعت‌ها در سندباکس ابری خودگردان (autonomous) کار می‌کند و معمولاً یک پول‌ریکوئست تحویل می‌دهد.

Jules · Copilot agent mode ·  
background agents

## در ترمینال

هدف را به زبان ساده می‌دهی؛ عامل در کل کدبیس کار می‌کند، ابزار و تست اجرا می‌کند و بر اساس نتیجه تکرار می‌کند.

Claude Code · Codex CLI ·  
Antigravity · Cline

## در ویرایشگر

تکمیل درون‌خطی، پنل گفت‌وگو، آگاهی از کل کدبیس. جایی که کار «در جریان» می‌ماند.

Copilot · Cursor · Windsurf

**انتخاب درست:** به ماهیت کار بستگی دارد، نه به اینکه کدام دسته روی نردبان خودگردانی بالاتر است. بیشتر توسعه‌دهندگان هر سه را در یک روز به کار می‌گیرند.

## مسئله ۸۰ درصد

عامل‌ها حدود ۸۰٪ کد یک ویژگی را به سرعت تولید می‌کنند. ۲۰٪ باقی‌مانده – حالت‌های مرزی، مدیریت خطا، نقاط یکپارچگی و الزام‌های ظریف درستی – دانش زمینه‌ای عمیقی می‌خواهد که مدل‌های امروز اغلب ندارند.

**ماهیت خطاها عوض شده:** از اشتباه نحوی به شکست مفهومی – فرض غلط درباره منطق کسب‌وکار، نپرسیدن درباره نیازمندی مبهم، جافتادن حالت مرزی، و تصمیم معماری‌ای که بار نگه‌داری بلندمدت می‌سازد. این خطاها سخت‌تر دیده می‌شوند، دقیقاً چون کد «درست به نظر می‌رسد» و حتی ممکن است از تست‌های ساده رد شود.

**وضعیت درست:** هوش مصنوعی را برای آنچه خوب انجام می‌دهد به کار بگیرید (پیاپی‌سازی سریع کارهای خوش‌تعریف) و توجه خودتان را برای آنچه در آن ضعیف است نگه دارید (نیازمندی مبهم، موازنه معماری، راستی‌آزمایی درستی).

# وقتی خودِ محصول یک عامل است

تا اینجا از عامل‌ها برای ساختن نرم‌افزار استفاده کردیم. اما اگر چیزی که باید بسازید، خودش یک عامل باشد چه؟ ربات پشتیبانی که درخواست بازپرداخت را رسیدگی می‌کند؛ دستیار پژوهشی که منابع را تطبیق می‌دهد؛ ابزار داخلی که انطباق را پایش می‌کند.

```
# one-time setup
uvx google-agents-cli setup

# then, inside your own coding agent:
> Build a support agent that answers questions from our docs.
> Evaluate it on the FAQ dataset.
> Deploy it to Agent Engine.
```

پشت همین سه جمله: ساخت اسکلت پروژه، نوشتن کد، تولید مجموعه ارزیابی، اجرای آن، دیپلوی و گزارش نتیجه. نمونه اولیه دیروز روی لپ‌تاپ، امروز بدون بازنویسی به عامل تولیدی بدل می‌شود.

# اقتصاد توسعه مبتنی بر هوش مصنوعی

گفت‌وگو معمولاً با «سرعت توسعه‌دهنده» شروع و تمام می‌شود. اما سنجۀ مهم‌تر برای رهبران فنی، هزینه کل مالکیت است.

# بدهی پنهان کدنویسی شهودی

سرمایه‌گذاری اولیه تقریباً صفر است؛ اما بار عملیاتی انباشته می‌شود.

## جبران امنیتی

بدون مهار ارزیابی خودکار، تولید سریع کد یعنی تولید سریع آسیب‌پذیری. هزینهٔ رفع نقص امنیتی در تولید، نمایی بیشتر از رفع آن در طراحی است.

## مالیات نگهداری

کدی که بی‌انسجام ساختاری نوشته شده؛ شش ماه بعد، مهندس انسانی باید روزها صرف مهندسی معکوس آن کند.

## نرخ سوخت توکن

ریختن فایل‌های حجیم و بی‌ساختار در کانتکست ویندو (context window) و درخواست مکرر اصلاح اشتباهات راستی‌آزمایی‌نشده: حلقه‌ای گران با نرخ موفقیت پایین در نوبت اول.

**ترجمهٔ مالی:** سرمایه‌گذاری اولیهٔ کم، هزینهٔ عملیاتی زیاد.

# سرمایه‌گذاری مهندسی عامل محور

## هزینه اولیه بیشتر

طراحی اسکیمای API، ساخت مجموعه تست قطعی، و مهم‌تر از همه ساختاردهی زمینه عامل – پیش از تولید نخستین خط کد تولیدی.

## هزینه نهایی کمتر

هزینه افزودن و نگهداشتن هر ویژگی به شدت افت می‌کند. عامل درون «کارخانه‌ای» حکمرانی شده کار می‌کند: خروجی از پیش تست شده و هم‌راستا با استانداردهاست.

**مهندسی زمینه (Context engineering) به عنوان اهرم مالی:** مدل‌ها بابت هر تکه اطلاعاتی که می‌فرستید پول می‌گیرند. فرستادن ریبوی ۱۰۰,۰۰۰ توکنی در هر درخواست، در مقیاس، اقتصادی نیست. زمینه متراکم و پرسبک‌نال، نرخ موفقیت نوبت اول را بالا می‌برد و حلقه‌های پرهزینه تست و خطا را حذف می‌کند.

# مسیریابی هوشمند مدل

| نوع کار                            | مدل مناسب                             | چرا                               |
|------------------------------------|---------------------------------------|-----------------------------------|
| نیازمندی، معماری، پیاده‌سازی اولیه | مدل بزرگ و پیشرو                      | استدلال پیچیده ارزش هزینه را دارد |
| تولید تست، بازبینی کد، پایش CI/CD  | مدل کوچک‌تر، سریع‌تر و بسیار ارزان‌تر | کار قطعی‌تر و کم‌پیچیدگی‌تر است   |

**ضدالگو:** در گردش کار شهودی، یک مدل عظیم برای همه‌چیز به کار می‌رود – حتی برای اصلاح یک غلط تایپی یا نوشتن یک تست واحد ساده. این یعنی پرداخت قیمت درجه یک برای کار درجه سه.

# ده خط که کیفیت خروجی عامل را عوض می‌کند

همین حالا برای ریپو خودتان یک `AGENTS.md` بنویسید – چهار بخش، هر کدام دو تا سه خط.

۱

## پشته فناوری

زبان، چارچوب، نسخه‌ها، ابزار ساخت.

۲

## قراردادها

سبک کد، ساختار پوشه‌ها، الگوی نام‌گذاری.

۳

## قواعد سخت

کارهایی که عامل هرگز نباید بکند.

۴

## گردش کار

فرمان تست، فرمان قالب‌بندی، انتظار از پیام کامیت.

**عادت کلیدی:** هر بار عامل کاری کرد که نباید، یک قاعده تازه اضافه کنید. فایل قاعده، حافظه نهادی تیم شماست.

# سه اصل ماندگار

۱

## ساختار مقیاس می‌گیرد، شهود نه

کدنویسی شهودی برای کاوش و نمونه‌سازی معتبر است. برای نرم‌افزاری که سازمان به آن تکیه می‌کند، انضباط عامل‌محور انتخابی نیست.

۲

## هوش مصنوعی فرهنگ مهندسی را تقویت می‌کند

ضریب نیرو است و هم قوت‌ها و هم ضعف‌های شما را بزرگ می‌کند.

۳

## نقش انسان در حال تحول است، نه کوچک‌شدن

مهارت‌های تعیین‌کننده از پیاده‌سازی به قضاوت جابه‌جا می‌شوند: از نوشتن کد به طراحی سامانه‌هایی که کد تولید می‌کنند.

تولید حل شده است. راستی‌آزمایی، قضاوت و جهت‌دهی، پیشه تازه‌اند.

# تکلیف تا جلسه بعد

1. AGENTS.md امروز را روی ریپو واقعی خود بکارید و دست کم سه کار با عامل انجام دهید.

هر بار عامل خطا کرد، یک قاعده اضافه کنید و کامیت کنید تا سیر تکامل فایل دیده شود.

2. سه کار واقعی از پروژه خود را روی طیف بنشانید و برای هر کدام سازوکار راستی آزمایی متناسب پیشنهاد دهید.

یک صفحه، تحویل در آغاز جلسه دوم.

3. مقاله ترجمه شده جلسه اول را کامل بخوانید.

جلسه دوم با فرض خواندن آن شروع می شود.

# ابزارهای عامل و تعامل پذیری

عامل چگونه با جهان بیرون حرف می زند؟  
فراخوانی تابع، پروتکل زمینه مدل (MCP) و گفت و گوی عامل با عامل.

پرسشی دارید؟ همین حالا بپرسید.